

"ThreeBlueOneBrown" and Khan Academy"

"Deep learning" ~ Ian Goodfellow, Joshua Bengio (Benzio?)

4 Computation :

Antecedent $\rightarrow$ Function $\rightarrow$ Consequent

$$y = f(x)$$

$\downarrow$
mapping function
or algorithm

☒ Hand Computing $\left\{ \begin{array}{l} \rightarrow \text{Precise Result} \\ \rightarrow \text{Steps are unambiguous} \\ \rightarrow \text{Steps are formally defined} \end{array} \right.$

☒ Soft Computing : Opposite of Hand Computing

- $\therefore$ More related to real life $\rightarrow$ bc life is pain
- $\rightarrow$ tolerance for imprecision and uncertainty
- $\rightarrow$ low solution cost
- $\rightarrow$ no mathematical modelling
- $\rightarrow$ may not yield precise solution
- $\rightarrow$ adaptive algorithm
- $\rightarrow$ works with uncertainty

Rules Thinking of a human:

1. Crossing a busy road
2. What movie to watch

#1

1. Check for a foot-overs bridge, or zebra crossing (on any alternatives)
2. Look ~~left~~^{right}, look ~~right~~^{left}, look right one final time then cross
3. Show/signal using hands if less time.
4. Check for others as if scared

#2.

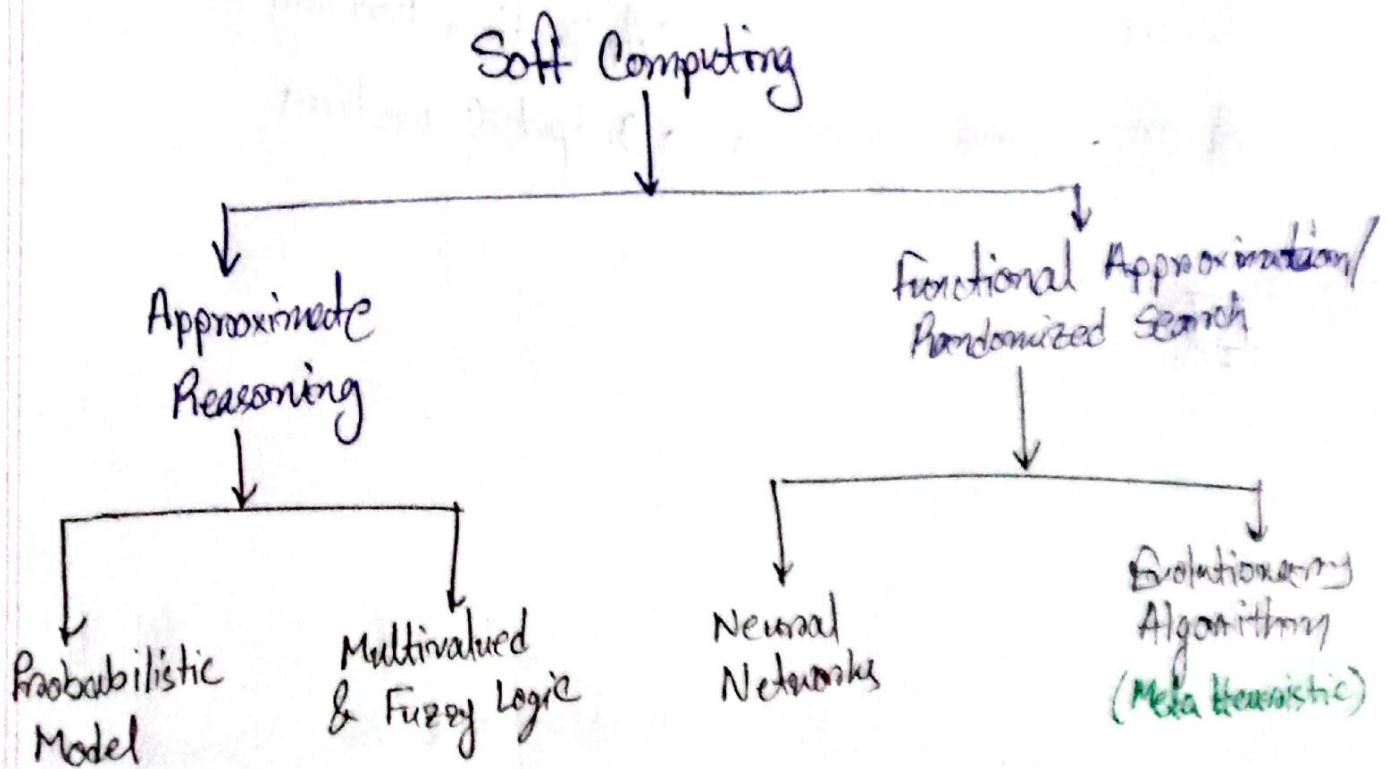
1. Find what genre everyone₁ is interested in (on majority)
2. What movies are ^{now} available

C-2, W-1 (Q-1)

24/08/26

* Hard computing works with exact data

Soft " " " " ambiguous and noisy data
↳ unnecessary



* aim is to make it think of ~~the~~ like human

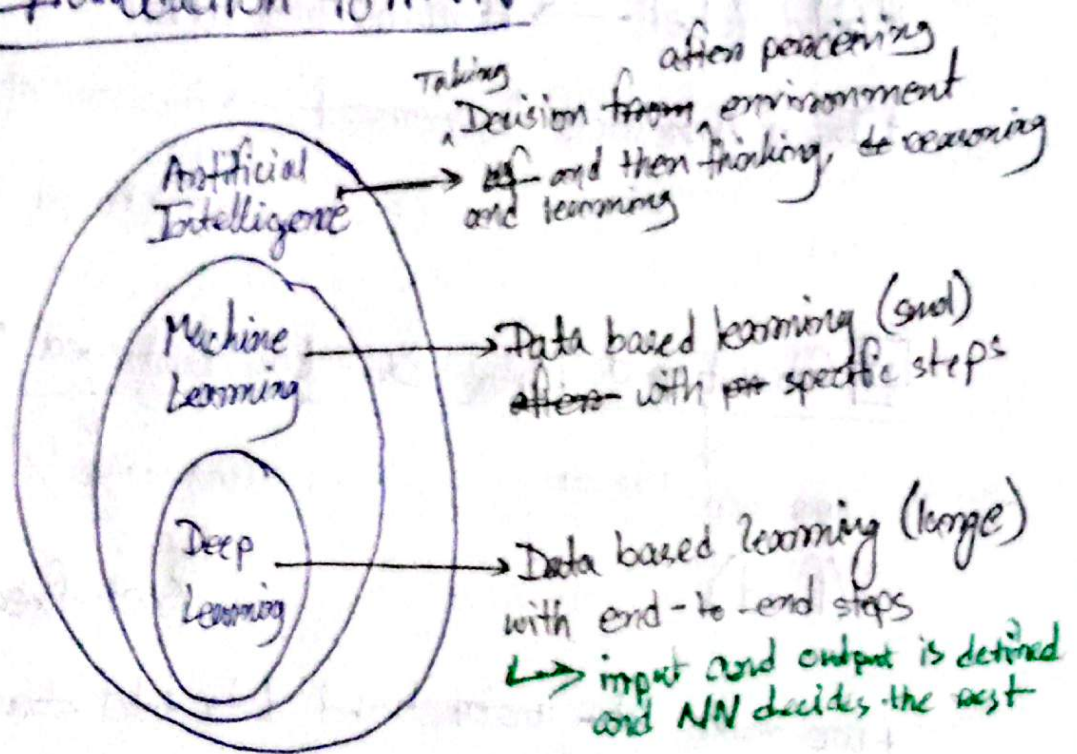
* defⁿs, characteristics & differences

Artificial Neural Network

Dendrite → Incoming Connection
Soma → Activation function
Axon → Output Connection

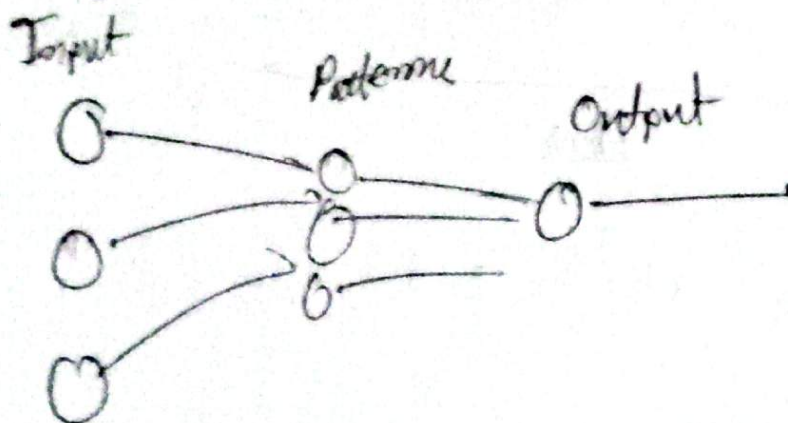


Introduction to ANN



- * Regression deals with continuous values
- Classification " " discrete "

Single Neural Network



C-3, W-1(Q-1)

25/06/26

* Gold label $\rightarrow$ Human Annotated

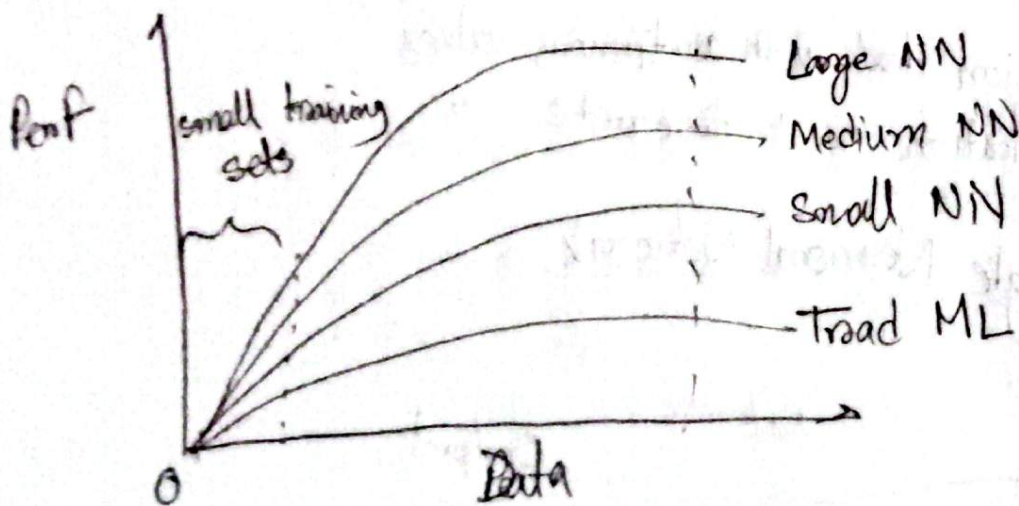
* Inter Annotators Agreement $\rightarrow$ Agreement among the annotators
 $\rightarrow$ Kappa Score

Structured Data & Unstructured Data

arranged ~~the~~
(fixed)

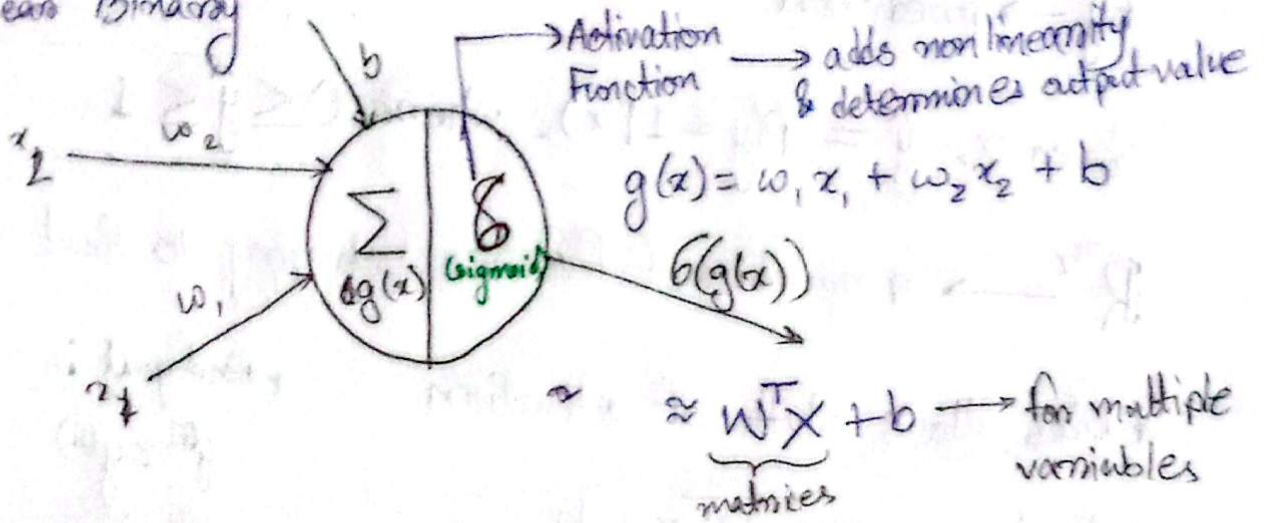
unarranged / different
types
(not fixed)

* we make ~~the~~ unstructured data into structured data before analyzing.



Logistic Regression: Single neuron

* Linear Binary



* weight $\perp$ Decision Boundary

$\rightarrow$ changing weight causes change in decision boundary

* weight changes rotates
 bias changes position/location

* dot and inner product are same

$$\hookrightarrow W^T X + b = W \cdot X + b$$

$$c=4, w=2(Q-1)$$

29/06/26

* Pixel Intensity $\rightarrow$ color intensity

* $\hat{y} \rightarrow$ prediction

* Precision:
cost functions, loss functions
help us to reach higher
precision

Given x , $\hat{y} = P(y=1|x)$, where $0 \leq \hat{y} \leq 1$

$\mathbb{R}^n \rightarrow$ a number of features belonging to Real Numbers

* bias allows shift of activation

* End goal is

$$\hat{y}^{(i)} \approx y^{(i)}$$

* logit $\rightarrow z = w^T x + b$

Cost Function & Loss Function (error)

Average of Loss Function

Error for a single training

* Squared Error Function $\rightarrow L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$

$\rightarrow$ forms abs. value

$\rightarrow$ cannot find local global minima
(non-convex)

$\rightarrow$ $\frac{1}{2}$ is added
for mathematical
convenience ($\frac{d}{dx}$)

$\rightarrow$ logarithm functions are convex

* Cross Entropy Loss $\rightarrow L(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}))$

* Cost Function $\rightarrow J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$

C-5, w-2(Q-1)

01/07/26

Given x (feature),

$$\hat{y} = P(y=1|x); \text{ where } 0 \leq \hat{y} < 1$$

For binary classification:

$$\therefore \text{MAX} \rightarrow P(y|x) = \hat{y}^y (1-\hat{y})^{1-y}$$

$$\text{for } y=0, P(y|x) = (1-\hat{y})$$

$$\text{" } y=1, P(y|x) = \hat{y}$$

by taking log on both sides, $\rightarrow$ maximum log likelihood

$$\log P(y|x) = \log [\hat{y}^y (1-\hat{y})^{1-y}]$$

$$= y \log \hat{y} + (1-y) \log (1-\hat{y})$$

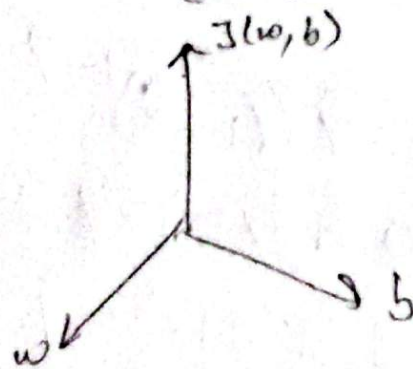
$\rightarrow$ By maximizing this we can ensure that x feature is optimized for y

We flip the signs to make it loss function to minimize,

$$\therefore L_{\text{CE}}(\hat{y}, y) = -\log P(y|x) = -y \log \hat{y} - (1-y) \log (1-\hat{y})$$

Gradient Descent: Optimization Algorithm

(to optimize weight and bias)



* Convergence $\rightarrow$ reaching maximum or minimum

* α = learning rate $\rightarrow$ steps of descent

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{dJ(w)}{dw}$$

$\downarrow$

For both,

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{\partial J(w, b)}{\partial w}$$

$$b_{\text{new}} = b_{\text{old}} - \alpha \frac{\partial J(w, b)}{\partial b}$$

C-6, W-2(Q-1)

02/06/26

Do we need Gradient Descent?

→ Yes → While Brute Force gives a faster answer (for shorter
fewer steps)

it goes through all of the values.

↓
Gradient Descent tests selective ones

* Narrow the problem → IF helps to search

Computation Graph:

Forward Pass → Backward Pass

↓
compute the
output

↓
compute the
weight gradient / updates weight
on propagate error

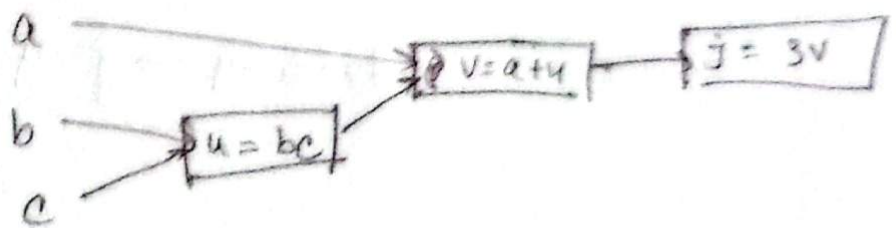
Let, $J(a, b, c) = 3(a + bc)$

↓

1. $u = bc$

2. $v = a + u$

3. $j = 3v$

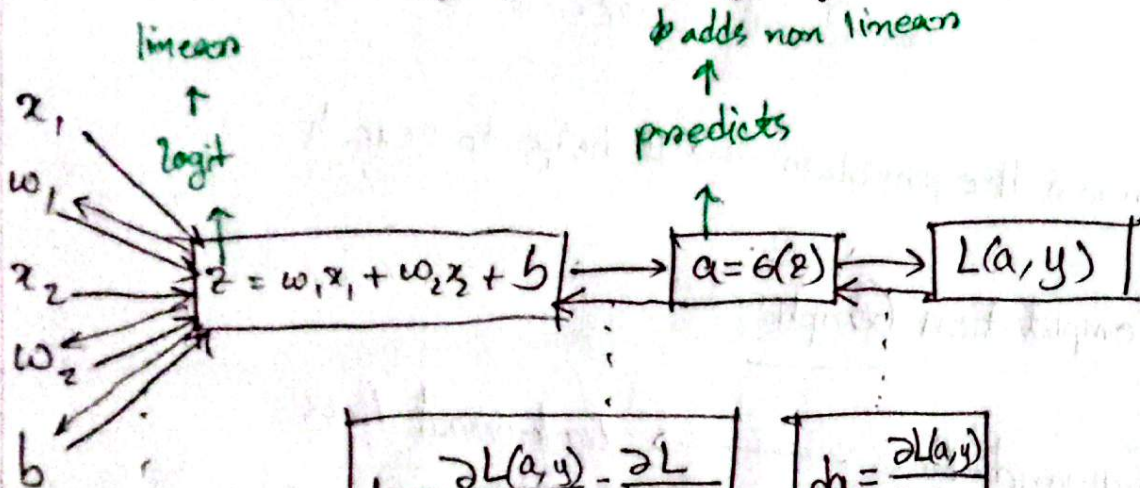


* Logistic Forward:

$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

$$L(a, y) = -(y \log(a) + (1-y) \log(1-a))$$



$$dz = \frac{\partial L(a, y)}{\partial z} = \frac{\partial L}{\partial z}$$

$$da = \frac{\partial L(a, y)}{\partial a}$$

$$= -\frac{y}{a} + \frac{1-y}{1-a}$$

$$dw_i = dz \cdot x_i$$

$$db = dz$$

$$= \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z}$$

+ we increase dim using

$$\therefore dz = a - y \rightarrow \hat{y} - y$$

$$\therefore w_i = w_i - \alpha dw_i$$

$$b = b - \alpha db$$

C-7, Q-2 (Q-1)

(C.1.1) 06/07/26

Logistic Regression on m examples: → nows

$$z^i = w^T x^i + b$$

for ~~each~~ iteration = 1 to ⁿ required → for n iterations

for $i = 1$ to m

forward pass $\rightarrow z(i), a(i), J$

backward pass $\rightarrow dz(i), dw_i(i), db$ for n features

avg $\rightarrow J/m, dw_i/m, db/m$

Gradient descent $\rightarrow w_i = w_i - \alpha * dw_i$

* but this is high $O(n)$ ~~complexity~~ complexity

$\therefore$ we make it easier using vectors → numpy does it cell by cell

$\downarrow$
np.dot (mat 1, vect 1, vect 2)

$$X = \begin{bmatrix} x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(m)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \dots & 1 \end{bmatrix} \rightarrow [w^T] \cdot \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ 1 \end{bmatrix}$$

* Reminders: n (Features) = n (weight)

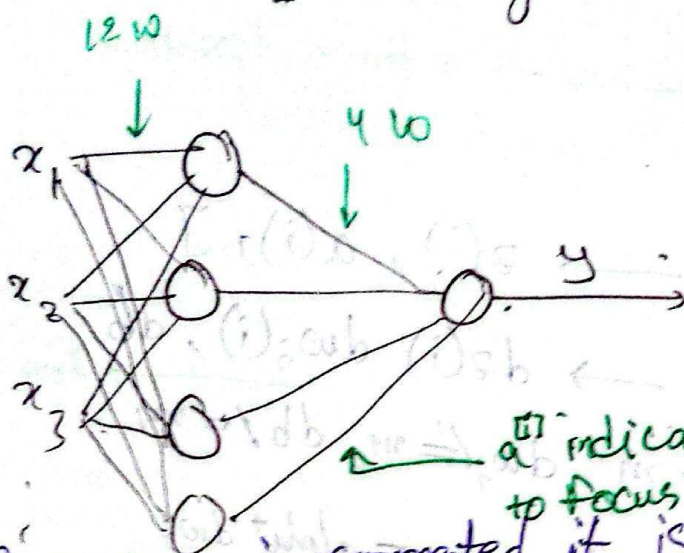
$$\begin{aligned} [z^{(1)} \ z^{(2)} \ z^{(3)} \ \dots \ z^{(m)}] &= w^T X + [b \ b \ \dots \ b] \\ &= [w^T x^{(1)} + b \quad w^T x^{(2)} + b \quad \dots \quad w^T x^{(m)} + b] \end{aligned}$$

C-8, W-3 (Q-1.5)

08/07/26

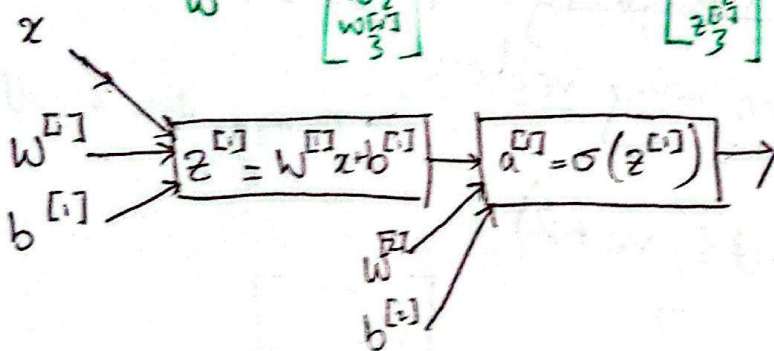
Shallow Neural Network

1 hidden layer $\rightarrow > 1$ is deep



* if everyone is connected it is dense network
 $a^{[i]}$ indicates which feature to focus

$$W^{[1]} = \begin{bmatrix} w_{11}^{[1]} \\ w_{21}^{[1]} \\ w_{31}^{[1]} \\ 3 \end{bmatrix} \rightarrow z^{[1]} = \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ 3 \end{bmatrix} \rightarrow a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \end{bmatrix}$$



$a_n^{[i]} \rightarrow$
 i = layer no.
 n = node number

* find matrix size from layers

C-9, W-3 (Q-1.5)

09/07/26

$$z_1 = w_1 \times x_1 + b_1$$

$$= (4 \times 3) \times (3 \times 1)$$

$$= (4 \times 1)$$

$$z = w \times x + b$$

$$z = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_1 \\ b_1 \\ b_1 \end{bmatrix}$$

$$= \begin{bmatrix} w_{11}x_1 + b_1 \\ w_{22}x_2 + b_1 \\ w_{33}x_3 + b_1 \\ w_{44}x_4 + b_1 \end{bmatrix}$$

$$= \begin{bmatrix} w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1 \\ w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_1 \\ w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_1 \\ w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_1 \end{bmatrix}$$

$$H^1, W^{[1]} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix} \text{ and } X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$z_1^{[1]} = w_1^{[1]T} X + b_1^{[1]}$$

$$z_2^{[1]} = w_2^{[1]T} X + b_2^{[1]}$$

⋮

$$a_1^{[1]} = \sigma(z_1^{[1]})$$

$$a_2^{[1]} = \sigma(z_2^{[1]})$$

⋮

where

$$W_1^{[1]} = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix}$$

$$W_2^{[1]} = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix}$$

⋮

$$z_1^{[1]} = [$$

$$z^{[2]} = W^{[2]} a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

Quiz Syllabus:

Soft computing + Logistic Regression [slide 42]

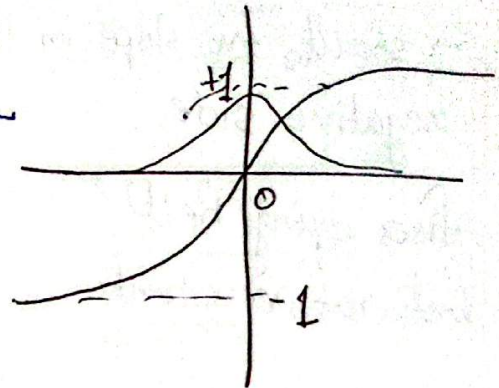
online
C-10, W-4? (Q-1.5)

16/07/26

Types of Activation Function:

* Hyperbolic Tangent Function:

- vanishing gradient still remains
- still computationally expensive



but,

- zero centered

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

* vanishing gradient causes
"one problem in deep network"

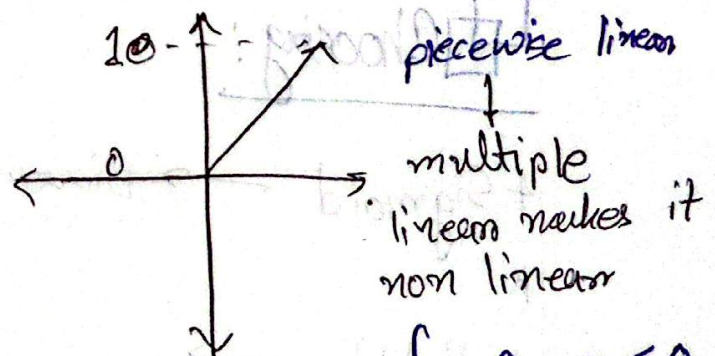
* ReLU (Rectified Linear Unit)

- const value so $\frac{d}{dx}$ exists = 1

- no vanishing gradient
but

dying ReLU exists

↳ if gradient = 0,
neuron shuts down

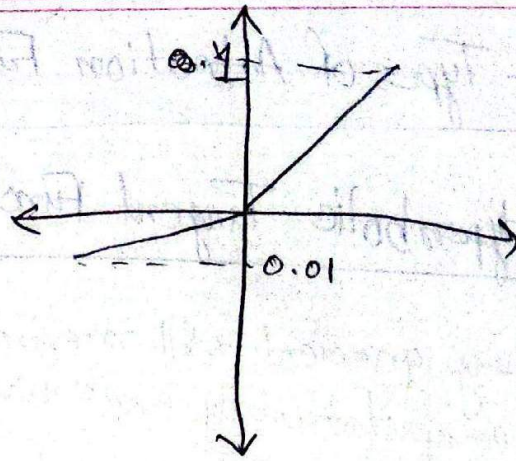


$$f(x) = \max(0, x) = \begin{cases} x=0 & x < 0 \\ x=x & x > 0 \end{cases}$$

* Leaky - ReLU

→ small +ve slope in the negative side

fixes dying ReLU
but not consistent



$$f(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

Choosing:

* Sigmoid → Binary Classifier → Output

*

* ReLU →

→ Hidden

* As a rule of thumb, better to start with ReLU

Soft Max: $a_i = \frac{e^{z_i}}{\sum_{k=1}^c e^{z_k}}$

where $\sum_{i=1}^c a_i = 1$

→ $\frac{\text{num}}{\text{sum of all}}$

* e is used as e^n makes it positive for any value of n

combination of multiple sigmoid

→ can be used in binary only on the output layer

Derivation:

*Review:

i) Logits $\rightarrow w^T x + b$

ii) Use of weight and bias $\rightarrow$ i) rotates and shifts
ii) weight influences features

iii) $n(\text{Features}) = n(\text{Weight})$

iv) parameters $\rightarrow$ vars that change during training

v) Loss vs Cost $\rightarrow$ Error $(\hat{y} - y)$ of single ^{sample} is Loss
Avg is Cost

vi) Loss func is better if convex $\rightarrow$ global minima = 1

vii) Derivative in Grad. Descent $\rightarrow$ Slope of a function at a point

viii) Derivative of activation function

ix) $(0, 1) \rightarrow$ for output

Rest is ~~us~~ for hidden layer

Cross Entropy Loss

42

* Logits is not probability
↳ linear ^{continuous} value

* ~~no~~ Softmax > Std Normalization
because Std-Norm has similar ratio
& Softmax amplifies

Cross Entropy Loss
↓
measure of error

$$H(x) = \begin{cases} - \int_x p(x) \log(x) \\ - \sum_x p(x) \log p(x) \end{cases} \rightarrow \text{physics}$$

↳ log because to prevent close to 0
- because log gives -ve value
multiplied to have some value

∴ For DL,

$$L_{CE} = - \sum_{i=1}^n t_i \log(p_i) \quad \text{for } n \text{ classes}$$

$\uparrow$ Predicted
 $\downarrow$ true value

For the slide

$$= - \{ (1) (\log_2(0.775)) + (0) (\log_2(0.16)) + (0) (\log_2(0.039)) + (0) (\log_2(0.007)) \}$$

C-1/4, W-5 (Q-2)

23/07/21

α $\rightarrow$ Optimization Variation

* Basic $\rightarrow$ Grad, Stochastic GD

Prev. ^{Dom.} $\rightarrow$ AdaGrad, RMS Prop

Cutro $\rightarrow$ Adam

* Basic Grad. Desc. $= W_{t+1} = W_t - \alpha \frac{\partial L}{\partial W}$

$\rightarrow$ But for a straight line, it gets stuck

$\hookrightarrow$ we need momentum to move forward

$\downarrow$
this is why we use Momentum Based GD

$\rightarrow$ GD works on loss of the whole data (cross)

$\hookrightarrow$ but this causes some time loss

$\hookrightarrow$ so, we do it one at a time using approximation

$\downarrow$
Stochastic GD $\rightarrow$ if we take a batch
Mini-Batch GD (?)

$\downarrow$
because the value will be different compared to the whole dataset

* 3D Graphs $\rightarrow$ Contour Graph

* All algorithms emerged from the movement of in contour graph

* Steep Curve = Gradient is large ϵ = Large Update

Gentle " = " " small = Slow Update

* SGD faces a problem as one by one update causes massive changes in the direction

↳ additionally, bias is dense as value is almost always constant
but weight can be 0 so it is sparse updates more
updates less

□ Momentum Based Gradient Descent: using previous updates to gain momentum/confidence

$$\text{update}_t = \underbrace{\gamma \cdot \text{update}_{t-1}}_{\text{history}} + \gamma \nabla w_t$$

$$w_{t+1} = w_t + \text{update}_t$$

+ Exponential ^{Moving} ~~Weighted~~ Average
= Take some discount to old values

* Too much velocity can create oscillation

Newton's Accelerated GD (NAG):

* dir first, grad later $\rightarrow$ Use historical data first then we find the gradient

$\rightarrow$ computationally expensive & slower $\rightarrow$ because two steps

$$w_{\text{look_ahead}} = w_t - \gamma \cdot \text{update}_{t-1}$$

$$\text{update}_t = \gamma \cdot \text{update}_{t-1} + \eta \nabla w_{\text{look_ahead}}$$

$$w_{t+1} = w_t - \text{update}_t$$

GD with Adaptive Learning:

Decay the learning rate in proportion to update history

* dense features
update more
sparse features
update less

AdaGrad:

$$v_t = v_{t-1} + (\nabla w_t)^2 \rightarrow \text{dense feature} = \text{higher grad}$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

$\rightarrow$ to make non 0
 $\rightarrow$ to " v_t smaller

* Aggressive decay = small update for frequent parameters

Root Mean Square Propagation (RMSProp):

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla \omega_t)^2$$

$$\omega_{t+1} = \omega_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla \omega_t$$

Adaptive Momentum (Adam):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla \omega_t \leftarrow \text{adapt momentum}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla \omega_t)^2 \leftarrow \text{adaptive learning}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

* usually $\beta_1 = 0.9$
 $\beta_2 = 0.999$

$$\omega_{t+1} = \omega_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

+ Bias Correction to prevent bias in eqⁿ

↳ bias $\neq b$

bias in this case means biased towards b & ω

Self-Study: Pros and Cons of Adaptive Learning and ADAM

+ Bias Connection & Mathematical Formula

Bias - Variance Tradeoff

* We want a generalized state $\rightarrow$ Not over or underfitting

$\rightarrow$ (accuracy)

* Bias $\rightarrow$ dist from ϕ actual } we want low bias
 Variance $\rightarrow$ ^{how} dispersed from actual } low variance

$\rightarrow$ (precision)

* Too many features = curse of dimensionality

* Point of Bias - Variance Tradeoff
 is Sweet Spot

[Pattern slide]

$\downarrow$
 C-3, W-1, Q-1
 24/06/26
 Pattern

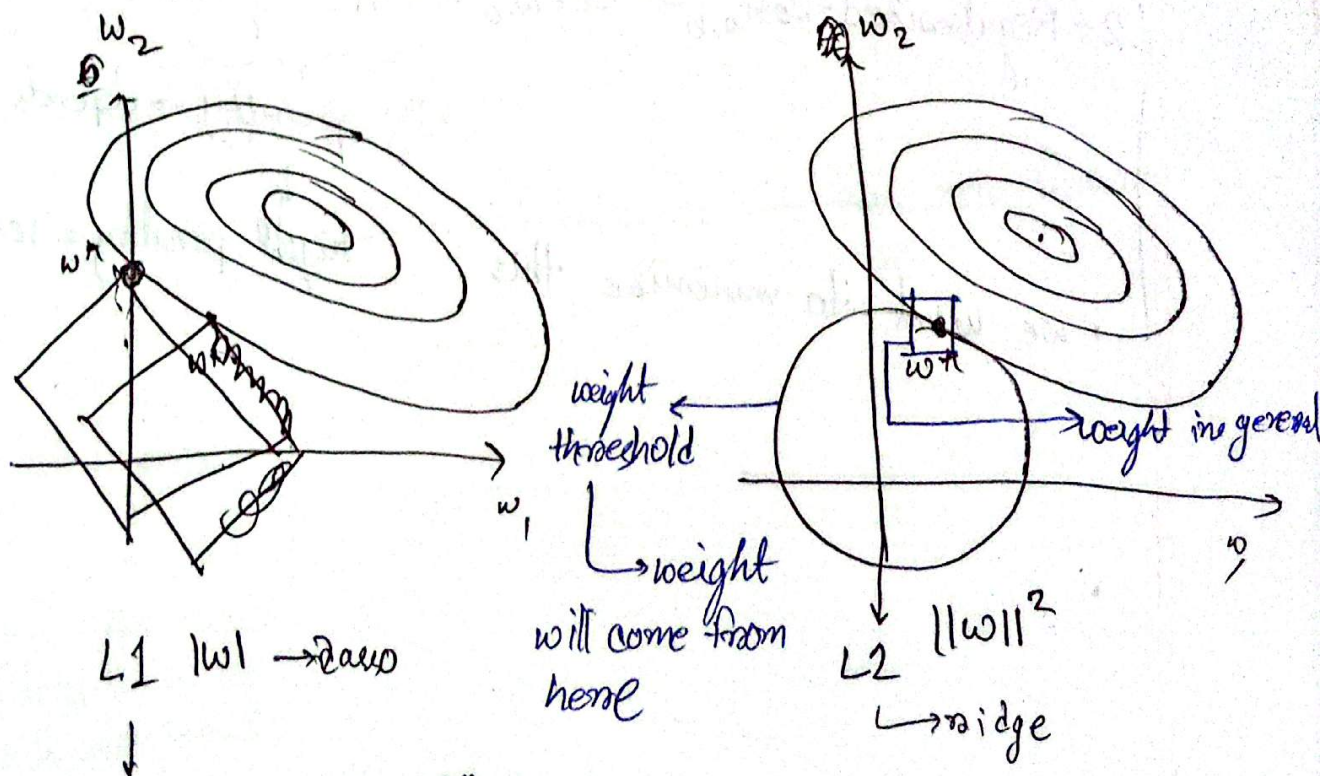
* Data Hungry $\rightarrow$ Feeding data gets
 more accuracy

C-17, 18, W-7 (Q-2)

04/08/26

* Overfitting is caused by $w \rightarrow$ higher weight
= more importance
leads to ~~more~~ fitting too specific

∴ we use regularization $\rightarrow$ constraints weights
 $\hookrightarrow$ intentionally making training harder
so testing goes better
 $\hookrightarrow$ like ISC



makes one point (weight)

Zero $\rightarrow$ may turn off features

$\hookrightarrow$ sometimes this is used for feature reduction

* which is better? $\rightarrow$ self study

Early Stopping: Stopping when $| \text{training} - \text{validation} | \text{min}$

Penalty:

for L2:

$$\text{Cost}_{w,b} = \frac{1}{n} \sum_{i=1}^n L(y^i, \hat{y}^i)$$

$$\text{L2-Regularized-Cost}_{w,b} = \text{Cost}_{w,b} + \underbrace{\frac{\lambda}{n} \sum_j w_j^2}_{\text{penalty} \rightarrow \text{depends on weight}}$$

hyperparameters that decides the intensity of penalty

→ we want to minimize this

high penalty = less weight

Dropout: prob of turning off nodes

CNN → originally for images
↳ CNN → RNN → LSTM → Transformers

* Use a filter to find a specific value

$$f(x) \otimes g(x)$$

↳ convolution → finds new feature map

Input Layers →

→ Since this causes under/overfitting we add ~~more~~ more kernel

* CNN → learns kernel / filter value with weight

* Unlike a ^{usual} dense network, each node only interacts with the filter → That's why it's sparse

* Usually this can cause information loss

↳ but for CNN, in later layers they are influenced
* Same weight for same kernel, ^{weight sharing} → we take diff weight for another kernel

C-10, W-8, A-2

17/08/26

□ CNN → both extracts and classifiers

↓ * features in slide = filter

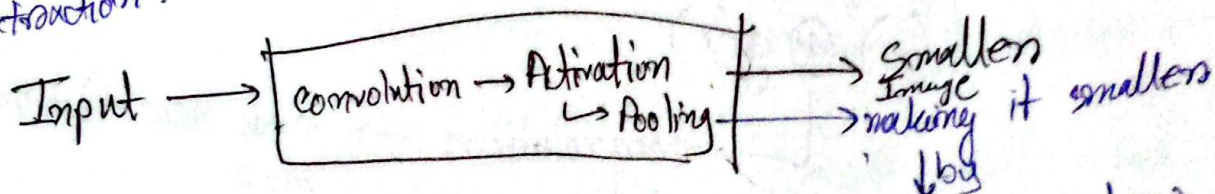
applying filters → convolution

getting matrix using convolution → feature maps

n(f)

n(filters) = n(feature maps)

Extraction: Combination of



↳ The smaller the image the better the output

↓ by
that sliding window
and taking highest
small stride for diverse
large " " important

Classification

Smaller Image → Fully Connected NN → Output

* ReLU → removes -ve value
& adds non linearity

* If swapping columns gives useful info → we ^{can't} use CNN

$$\text{CNN Output Shape} = \left\lfloor \frac{(W \ominus F + 2P)}{S} + 1 \right\rfloor$$

$$\text{No. of param} = (m \cdot n + 1) \cdot k$$

w = width of image / feature
F = Filter size
P = Padding
S = size

#1. # Questions.

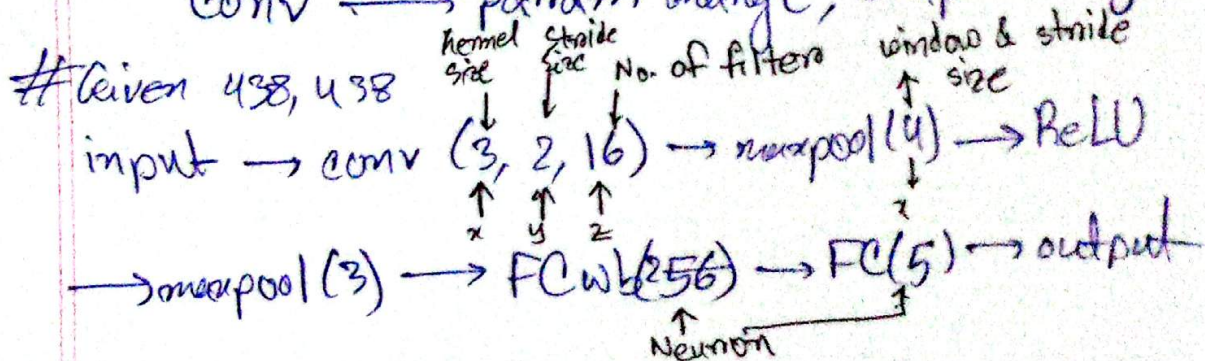
FCwB $\rightarrow$ (no. of neurones $\times$ shape) + shape

FC $\rightarrow$ (input $\times$ output)

~~no~~ dropout & activation $\rightarrow$ no change

pooling $\rightarrow$ shape change, no param change

conv $\rightarrow$ param change, shape change



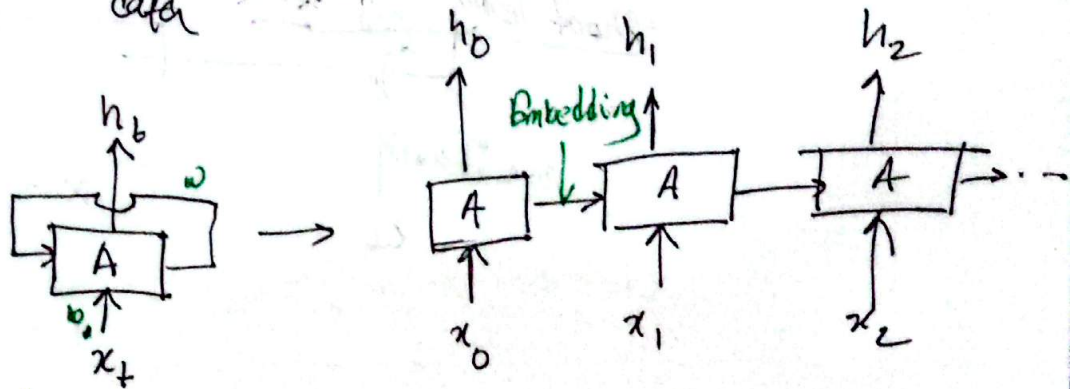
Layers	Output Shape	Param
Input	438, 438	0
Conv(3, 2, 16)	$\begin{matrix} w & h & \text{depth} \\ 218, & 218, & 16 \end{matrix}$ $\left\lceil \frac{218 - 4 + 2(0)}{4} \right\rceil + 1$	$((m \times n \times d) + 1) \times k = ((3 \times 3 \times 1) + 1) \times 16$ $= 160$ Nothing happened kernel/filter size of kernel prior filter is grayscale
max pool (4)	54, 54, 16	0
Relu	54, 54, 16	0
maxpool (3)	18, 18, 16	0
ReLU	18, 18, 16	0

<u>Layers</u>	<u>Shape</u>	<u>Param</u>
FCwB(256)	256, 256	$(18 \times 18 \times 16 \times 256) + 256$
FC(5)		256×5

RNN / LSTM

* Sequential data $\rightarrow$ current state depends on previous data

RNN:



* Same weight

$$\left. \begin{aligned} * h_t &= \tanh(W_{nh} h_{t-1} + W_{xh} x_t) \\ y &= W_{hy} h_t \end{aligned} \right\} h_t = f_w(h_{t-1}, x_t)$$

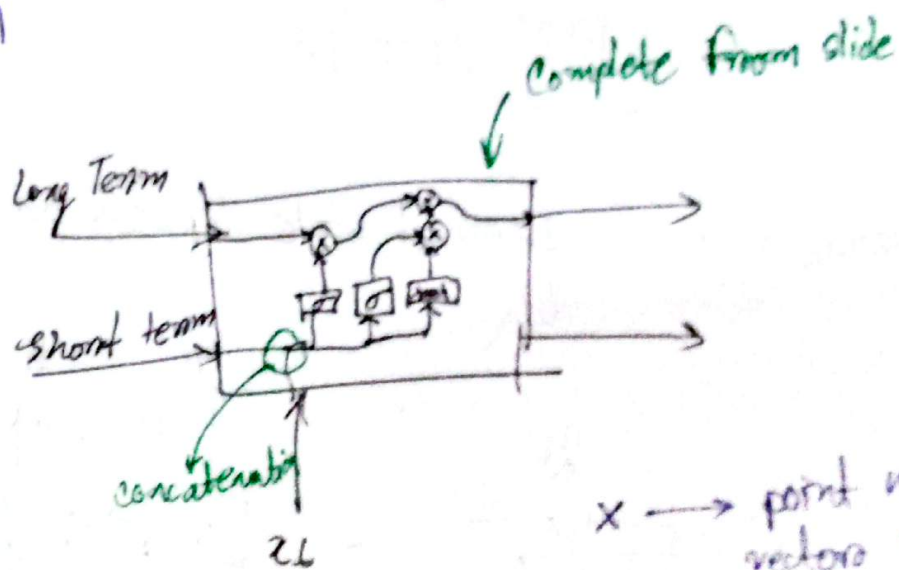
Backpropagation through time

* Stop-word-Removers $\rightarrow$ Remove unnecessary words

* However, sometimes for long sentences, values get lost
 $\rightarrow$ because there is no memory $\rightarrow$ we only pass Embedding

* Vanis suffers from both Vanishing and Exploding Gradient

1 LSTM



$\times$ $\rightarrow$ point wise
vector multiplication

$+$ $\rightarrow$ point wise
vector addition

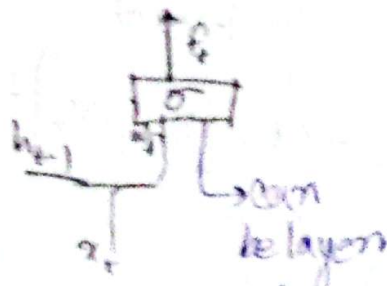
3 GATE $\rightarrow$ i) Forget Gate
ii) Input Gate
iii) Output Gate

* all dims inside are same

C-21, W-8, Q-2

20/08/26

Forget Gate:



$$f_t = \sigma(w_f [h_{t-1}, x_t] + b_f)$$

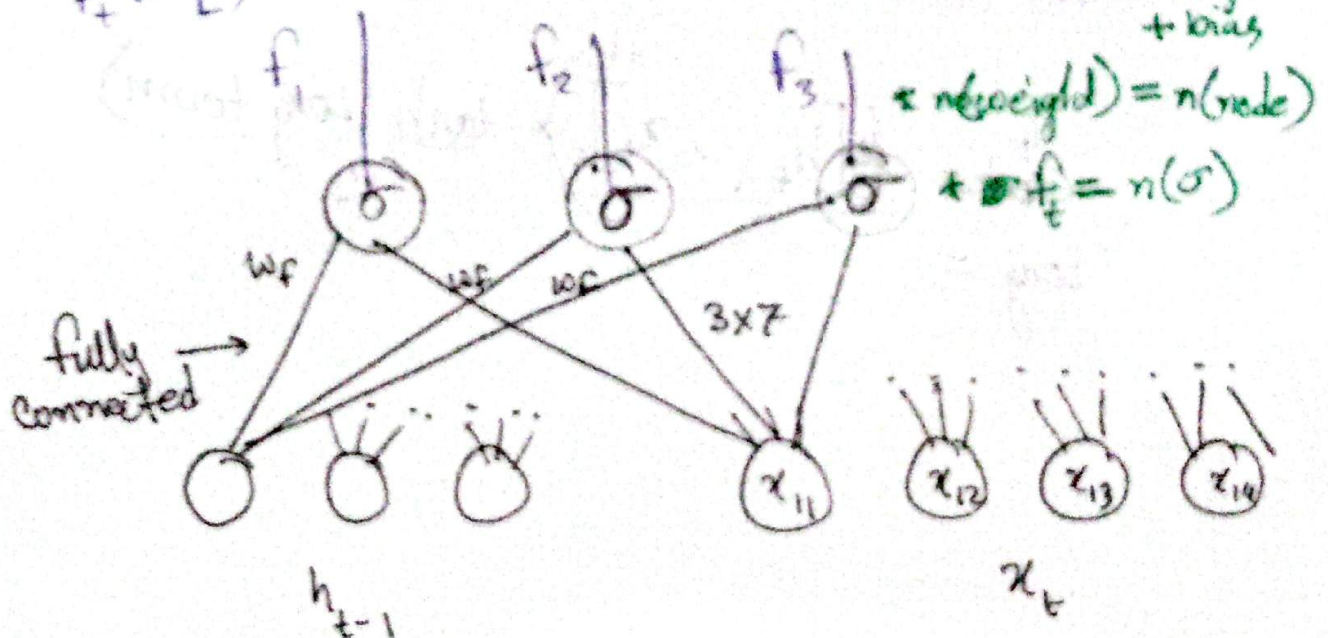
nodes are defined by mem.

Let, input is 4 dim: $x_{11}, x_{12}, x_{13}, x_{14}$ (4x1)

and 3 σ layers: $\sigma_1, \sigma_2, \sigma_3$

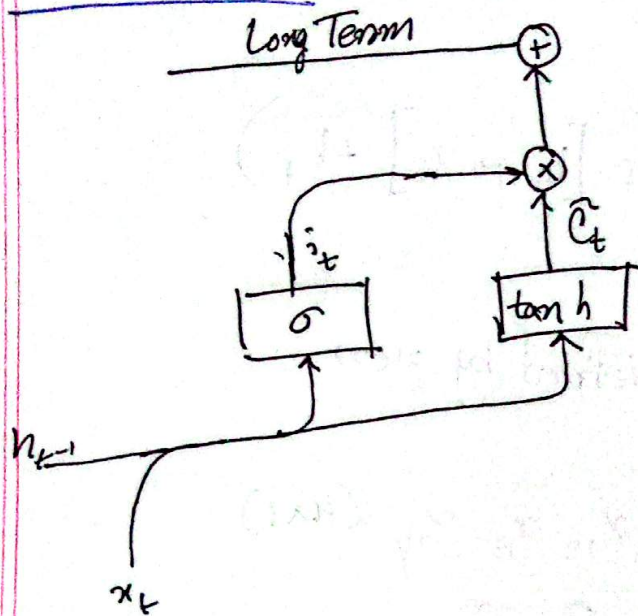
if $f_t = [0, 0, 0] \rightarrow$ forget everything
 $f_t = [1, 1, 1] \rightarrow$ remember it

type numbers = 3
 con of
 + parameter = no. of
 weight
 + bias



$n(\text{weight}) = n(\text{node})$
 $n(\sigma) = n(\sigma)$

* Input Gate: how much of input to take



* importance is learnt

$$i_t = \sigma(w_i [h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(w_c [h_{t-1}, x_t] + b_c)$$

* Output Gate:

$$h_t = \left(\frac{1}{2} [h_{t-1}, x_t] + b_o \right) \times \frac{\tanh(\text{Long term})}{c_t}$$

long =

GRU

* If LSTM has 3 gates and so is slower
→ that is why we remove gates

$$r_t = \sigma(w_r s_{t-1} + U_r x_t + b_r)$$

$$z_t = \sigma(w_z s_{t-1} + U_z x_t + b_z)$$

$$\hat{s}_t = \cancel{s_t} = \cancel{z_t \circ s}$$

$$\tilde{s}_t = \phi(w(r_t \circ s_{t-1}) + U x_t + b)$$

$$s_t = z_t \circ s_{t-1} + (1 - z_t) \circ \tilde{s}_t$$

C-22, W-9, Q-3

27/08/26

NLP & Word Embedding

Word Embedding : Text to Vectors

↳ way of representing words
to give the model a possibility to automatically
understand analogies

* TF-IDF, Bag of Words and One-hot representations
can make words into a vector

But they fail at semantic analysis

→ Word embeddings can help to understand analogies

* One-hot

Man

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

Woman

$$\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

To check if two vectors are similar
dot we use dot product

Sp

These words are related
but there is no mathematical
relationship

* Sparse Vectors:
A lot of 0

→ Language Model: ~~Some~~ The model which can predict next word or token

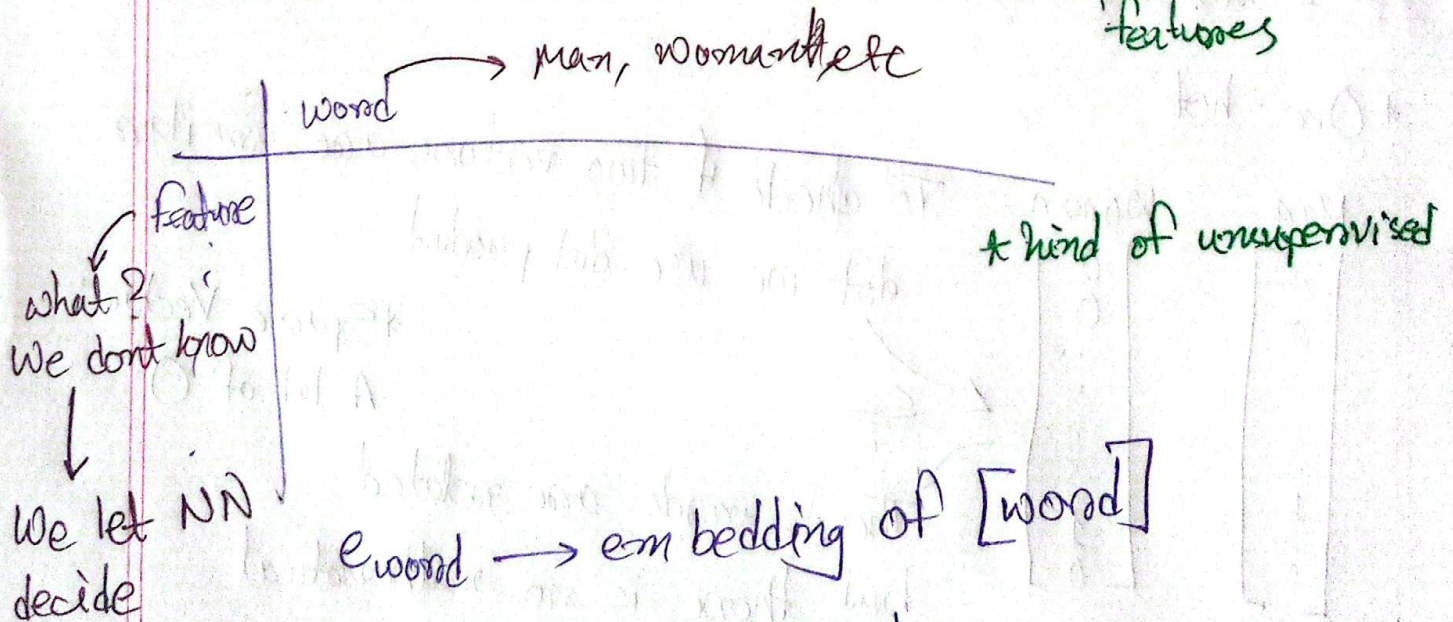
↳ based on relationship

(i) I want a glass of ^{orange} _____ [juice]

(ii) " " " " apple → will predict juice if trained on (i)

∴ One hot is bad because it is sparse (so makes it painful)

∴ We use Featurized Representation! → we add 'features'



$\text{dim} = \text{feature} \times \text{vocabulary}$

↑
we give numbers but it's trained by itself

→ tSNE Algo of words creates a similar pattern
↓
feature reduction

→ Analogy Reasoning → can connect words

$$e_{\text{man}} - e_{\text{woman}} \propto e_{\text{king}} - e_w$$

$$\therefore e_w = \underbrace{e_{\text{king}} - e_{\text{man}} + e_{\text{woman}}}_{\text{Difference of king from man but equivalent to woman}}$$

Difference of king from man
but equivalent to woman

→ Similarity

* To find specific embedding matrix we multiply it of its
using one-hot vectors

→ Transfer learning

→ word to vec